# Fonctions, Defer, Pointeurs et Mémoire

---

## Objectifs

1. Écrire des fonctions à retours multiples, variadiques (`...T`) et exploiter les fermetures lexicales (*Closures*) pour maintenir un état isolé en mémoire.
2. Maîtriser l'ordre d'exécution LIFO de `defer`, déjouer le piège de l'évaluation immédiate des arguments et implémenter le pattern de télémétrie `defer trace()()`.
3. Manipuler les pointeurs sans arithmétique et utiliser l'analyseur d'échappement du compilateur (`-gcflags="-m"`) pour prouver quelles variables restent sur la **Stack** ou s'échappent vers la **Heap**.

---

## Exercice 1 : Pipeline de calcul variadique et Fabrique de Closures

Vous développez les briques de calcul d'un agent de supervision chargé d'évaluer des métriques de consommation CPU et de déclencher des alertes en cas de dépassements répétés.

1. **Fonction variadique et retours multiples :**
   - Écrivez une fonction `analyserMetriques(valeurs ...float64) (min float64, max float64, moyenne float64, valide bool)`.
   - Si aucun argument n'est fourni (`len(valeurs) == 0`), la fonction doit retourner `valide = false` et les *Zero Values* pour les métriques.
   - Testez cette fonction de trois manières : sans aucun argument, avec une liste d'arguments directs (`analyserMetriques(12.5, 85.0, 42.3)`), puis en dépaquetant une slice existante `releves := []float64{45.2, 92.1, 18.4, 67.8}` grâce à l'opérateur `...`.
2. **Fabrique de Closures à état isolé (*Stateful Closure*) :**
   - Écrivez une fonction `creerDetecteurSeuil(seuilMax float64) func(valeur float64) (int, bool)` qui retourne une fonction anonyme.
   - La fonction retournée doit capturer un compteur interne (initialisé à `0` dans `creerDetecteurSeuil`). À chaque fois qu'on l'appelle avec une `valeur > seuilMax`, elle incrémente ce compteur interne et retourne `(compteur, true)`. Si `valeur <= seuilMax`, elle retourne `(compteur, false)` sans l'incrémenter.
   - Dans `main()`, instanciez deux détecteurs indépendants : `alerteCPU := creerDetecteurSeuil(80.0)` et `alerteRAM :=`

`creerDetecteurSeuil(90.0)`. Alimentez-les avec plusieurs valeurs et vérifiez que leurs compteurs internes sont totalement isolés en mémoire.

---

## Exercice 2 : Télémétrie avec `defer`, pile LIFO et piège d'évaluation

Vous devez instrumenter une fonction critique simulant une transaction bancaire afin de garantir la libération des ressources dans le bon ordre et de journaliser automatiquement son statut final ainsi que sa durée d'exécution.

1. **Le pattern de chronométrage en une ligne (`defer trace()()`) :**
   - Écrivez une fonction `mesurerExecution(nomOp string) func()` qui affiche `[START] <nomOp>` immédiatement lorsqu'elle est appelée, capture `debut := time.Now()`, et retourne une fonction anonyme `func()` qui affiche `[END] <nomOp> (Durée : <temps écoulé>)`.
2. **Simulation de transaction et piège de l'évaluation immédiate :**
   - Créez une fonction `executerTransaction(montant float64)` qui commence par activer le chronomètre en une seule ligne avec `defer mesurerExecution("Transaction Bancaire")()`.
   - Déclarez ensuite une variable locale `statut := "INITIALISATION"`.
   - Ajoutez successivement trois instructions `defer` :
     1. Un `defer fmt.Println("[AUDIT DIRECT] Statut final :", statut)` (qui subira le piège de l'évaluation immédiate).
     2. Un `defer` utilisant une **closure anonyme** `defer func() { ... }()` qui affiche `[AUDIT CLOSURE] Statut final : <statut>`.
     3. Un `defer fmt.Println("[CLEANUP] Fermeture de la session sécurisée")`.
   - Après ces déclarations `defer`, simulez un travail avec `time.Sleep(50 * time.Millisecond)`, puis modifiez la variable `statut = "VALIDÉE"` (ou `"REJETÉE"` si `montant <= 0`).
3. Exécutez le programme et expliquez précisément :
   - Dans quel ordre exact les 4 instructions `defer` se sont exécutées à la sortie de `executerTransaction` (règle LIFO).
   - Pourquoi `[AUDIT DIRECT]` affiche `"INITIALISATION"` alors que `[AUDIT CLOSURE]` affiche bien `"VALIDÉE"`.

---

## Exercice 3 : Pointeurs et inspection de l'*Escape Analysis* (`Stack` vs `Heap`)

Sur d'autres langages on a souvent le réflexe d'utiliser des pointeurs partout. En Go, vous allez prouver par l'expérimentation comment le compilateur décide d'allouer une variable sur la **Pile (Stack)** ou sur le **Tas (Heap)** à l'aide de son analyseur d'échappement.

1. Dans un fichier dédié `escape.go` (ou dans votre `main.go`), écrivez les trois fonctions suivantes qui manipulent des pointeurs :
    - **`appliquerBonus(score *int, bonus int)` :** prend un pointeur vers un entier et lui ajoute `bonus` sur place (`*score += bonus`), sans rien retourner.
    - **`creerScoreInitial(valeur int) *int` :** déclare une variable locale `s := valeur` et retourne son adresse mémoire `&s` à l'appelant (ce qui serait un bug critique de *dangling pointer* en C !).
    - **`creerScoreValeur(valeur int) int` :** déclare `s := valeur` et retourne directement la valeur `s`.
2. Dans `main()`, appelez ces trois fonctions :
    - Créez une variable locale `monScore := 100`, passez son adresse `&monScore` à `appliquerBonus(&monScore, 50)`.
    - Appelez `ptrScore := creerScoreInitial(200)` et déréférencez-le pour modifier sa valeur `*ptrScore += 10`.
    - Appelez `valScore := creerScoreValeur(300)`.
    - *(Important pour le test : n'utilisez pas `fmt.Println` dans un premier temps sur vos variables, car `fmt.Println` utilise des interfaces `any` qui faussent l'analyse d'échappement ; utilisez simplement `println(monScore, *ptrScore, valScore)` qui est une fonction interne du compilateur).*
3. **Inspection avec le compilateur Go :**
    - Lancez la compilation avec le flag d'analyse d'échappement :

```shell
go build -gcflags="-m"
```

    - Analysez la sortie du compilateur ligne par ligne et répondez aux questions suivantes :
        1. Dans `appliquerBonus`, que signifie l'annotation `score does not escape` ? La variable `monScore` de `main()` a-t-elle été allouée sur la Stack ou sur la Heap ? Pourquoi ("Sharing down") ?
        2. Dans `creerScoreInitial`, pourquoi le compilateur indique-t-il `moved to heap: s` ("Sharing up") ?
        3. Ajoutez maintenant un `fmt.Println(&valScore)` à la fin de `main()` et relancez `go build -gcflags="-m"`. Que devient `valScore` et pourquoi ?